

EDGAR Filing Download & Extraction Pipeline

Marco Zhang

March 2026

1 Introduction

This document describes the design and implementation of a production pipeline to download and extract SEC filings (10-K, 10-Q, 10KSB, 10QSB) for all firms across all years (1996–present). The pipeline is designed to run on the Mercury computing cluster at UChicago Booth, with local development and testing.

Scope

- **Filing types:** 10-K, 10-Q, 10KSB, 10QSB (no amendments or 8-K)
- **Coverage:** All CIKs in EDGAR, 1996–present (30 years)
- **Output:** Per-filing JSON (item-level text) and consolidated Parquet (one file per type $\times$ fiscal year)

Why a new pipeline?

The upstream `edgar-crawler` tool works well for small-scale extraction but has limitations for production-scale use:

1. **No year-level parallelism:** The original `main()` processes all years sequentially in one process. On a cluster with SLURM, we want one job per year running in parallel.
2. **No resumability:** If a download is interrupted, there is no clean way to detect which filings are missing and resume.
3. **Interactive prompts:** `download_indices()` prompts “Retry (Y/N)” on failure—this hangs in batch mode.

4. **Output organization:** The original organizes extracted JSON by filing type only. We need organization by `period_of_report` year for efficient downstream access (e.g., “give me all 10-K MD&A sections for fiscal year 2020”).

2 Design Decisions

2.1 Filing date vs. period of report

EDGAR indices are organized by *filing date*—the date the firm submitted the document. But researchers care about *period of report*—the fiscal period the filing covers. A 10-K filed on 2024-02-28 with `period_of_report = 2023-12-31` is a fiscal year 2023 filing.

The problem: `period_of_report` is not in the EDGAR index. It is only available after crawling each filing’s HTML index page (the `crawl()` function in `download_filings.py`, line 525–527). This means:

- **Progress tracking** uses filing-date year (known before download)
- **Output organization** uses period-of-report year (known after download)

This creates a slight asymmetry: a “download year 2023” job will produce output files scattered across period-of-report years 2022 and 2023 (since Q1 filings in Jan–Feb 2023 often have POR in late 2022). This is acceptable because each extracted JSON lands in the correct POR-year folder regardless of which download job produced it.

2.2 Reuse vs. rewrite

We vendor the core functions from `edgar-crawler` rather than rewriting them:

- `download_indices()`: Downloads EDGAR quarterly index files
- `get_specific_indices()`: Filters index by filing type and CIK
- `crawl()`: Fetches filing HTML index, extracts metadata including `period_of_report`
- `download()`: Downloads the actual filing document
- `ExtractItems.extract_items()`: Full extraction pipeline (HTML strip → item parse → JSON)

We apply targeted modifications (documented in `CHANGELOG.md`):

1. Make `DATASET_DIR` configurable at runtime via `set_data_dir()`
2. Remove interactive `input()` prompt in `download_indices()`
3. Extend filing type matching in `determine_items_to_extract()` and `extract_items()`: upstream uses exact equality (`== "10-K"`), changed to support variants (10-KT, 10KSB, 10QSB, 8-K/A). Done during tool exploration phase.
4. Convert all imports to relative package imports

2.3 JSON + Parquet dual output

We produce two output formats:

- **Per-filing JSON:** One file per filing, organized by type and period-of-report year. Useful for inspecting individual filings.
- **Consolidated Parquet:** One file per type $\times$ POR year. Columnar format with snappy compression enables selective column reads—e.g., loading only MD&A without all other items.

3 Pipeline Architecture

The pipeline has four sequential steps, each implemented as a standalone script that can be run independently or via SLURM job arrays.

3.1 Step 1: Build manifest (`build_manifest.py`)

Downloads all EDGAR quarterly index files and counts expected filings per type $\times$ filing-date year. This establishes the ground truth for progress tracking.

- **Input:** SEC EDGAR full index (fetched from `sec.gov`)
- **Output:** `Data/raw/indices/*.tsv` + `Data/raw/manifest.csv`
- **Runtime:** $\sim$ 30 minutes (one-time)

The manifest records how many filings of each type exist per year, so `check_progress.py` can report completion percentages.

3.2 Step 2: Download by year (`download_by_year.py`)

Downloads all filings of our four types for a single filing-date year. Designed for SLURM job arrays: `--array=1996-2025` runs 30 parallel jobs.

- **Input:** Index TSVs from Step 1
- **Output:** Raw filings in `Data/raw/filings/{type}/`, per-year metadata CSV
- **Progress:** `.running` $\rightarrow$ `.done` marker files in `Code/progress/`
- **Resumability:** On restart, reads existing metadata CSV, skips filings already on disk

For each filing, the script calls `crawl()` to fetch the HTML index page (extracting `period_of_report`, SIC, state, fiscal year end), then `download()` to fetch the actual document. Metadata is written atomically (via temp file + rename) after each successful download.

3.3 Step 3: Extract by year (`extract_by_year.py`)

Runs item extraction on all downloaded filings for a given filing-date year, organizing output by period-of-report year.

- **Input:** Raw filings + per-year metadata CSV from Step 2
- **Output:** `Data/extracted/json-per-filing/{type}/{por_year}/{filename}.json`
- **Key:** A filing filed in 2021 with `period_of_report=2020-12-31` lands in `10-K/2020/`

We call `ExtractItems.extract_items()` directly (not `process_filing()`) to control the output path ourselves. One subtlety: `items_to_extract` must be reset to an empty list before each filing's `determine_items_to_extract()` call. Otherwise, items from a previous filing type (e.g., 10-Q part-based items) persist and cause mismatches when extracting a different type (e.g., 10-K numbered items).

3.4 Step 4: Consolidate to Parquet (`consolidate_parquet.py`)

Converts all per-filing JSONs for a given type $\times$ period-of-report year into a single Parquet file.

- **Input:** Data/extracted/json-per-filing/{type}/{year}/*.json
- **Output:** Data/extracted/parquet-per-type-year/{type}/{year}.parquet
- Can run with `--all` to process every type/year combination, or target a specific pair

4 Project Structure

```
SEC_filings/
|-- Code/
|   |-- src/           # Pipeline scripts (Steps 1-4)
|   |-- utils/         # Vendored from edgar-crawler
|   |-- slurm/         # SLURM job submission scripts
|   |-- test/          # Local test suite
|   |-- progress/      # Year-level .running/.done markers
|   |-- logs/          # SLURM output logs
|   |-- CHANGELOG.md   # Modifications to vendored code
|   '-- README.md
|-- Data/
|   |-- raw/
|   |   |-- filings/{10-K,10-Q,10KSB,10QSB}/
|   |   |-- indices/
|   |   '-- manifest.csv
|   '-- extracted/
|       |-- metadata/FILINGS_METADATA_{year}.csv
|       |-- json-per-filing/{type}/{year}/
|       '-- parquet-per-type-year/{type}/{year}.parquet
|-- Docs/              # Research artifacts (plans, reports)
'-- DATA_DICTIONARY.md
```

5 SLURM Deployment

The pipeline runs on the Mercury cluster at UChicago Booth. Each step maps to a SLURM submission:

1. **Manifest** (interactive `srun`, one-time): Build the index and count expected filings.
2. **Download** (job array, `--array=1996-2025`): 30 parallel jobs, each downloading one year's filings. Wall time up to 7 days per job.

3. **Extraction** (job array, `--array=1996-2025`): 30 parallel jobs, each extracting items for one year. 8 GB memory per job.
4. **Consolidation** (single job): Converts all JSONs to Parquet. 16 GB memory.

Each SLURM script uses `--account=pi-<advisor>` for collaborator-tier access and `module load python/booth/3.12` for the cluster Python environment.

6 Verification

A local test suite (`Code/test/run_local_test.py`) validates the full pipeline with a minimal scope: year 2023, three CIKs (Apple, Microsoft, Nicholas Financial). The test verifies:

- Manifest creation and index download (4 quarterly TSVs)
- Filing download (12 filings: $3 \times 10\text{-K} + 9 \times 10\text{-Q}$) with metadata CSV and progress markers
- Resumability (re-run skips already-downloaded filings)
- Item extraction with correct period-of-report routing (filings filed 2023 with POR 2022 $\rightarrow$ 10-Q/2022/)
- 10-K JSON has 23 item columns; item_7 (MD&A) contains substantive text
- Parquet consolidation with selective column access

All tests pass. Run with:

```
python Code/test/run_local_test.py          # runs and cleans up
python Code/test/run_local_test.py --keep    # preserves test_data/
```